



Sistemas Embebidos I

Otoño 2026

Práctica 1: Inside the MCU

Andrea Ceja Lara

María Fernanda Conde Calatayud

19 / 08 / 2006

Índice

<i>Práctica 1: Inside the MCU</i>.....	2
Descripción de la práctica	2
Objetivos	2
Marco teórico	2
Materiales	3
Esquemático	3
Desarrollo	3
Resultados	4
Evidencias.....	6
Conclusiones	7

Práctica 1: Inside the MCU

Introducción

Descripción de la práctica

En esta práctica trabajamos con el Raspberry Pi Pico utilizando el simulador Wokwi, con el objetivo de comprender el funcionamiento de un blink. Para realizar la práctica desarrollamos cuatro códigos diferentes. Después de ejecutar cada uno de los códigos en Wokwi, observamos el comportamiento del LED y verificamos que el programa funcionara correctamente. Las señales obtenidas fueron posteriormente visualizadas mediante la herramienta Surfer, lo que nos permitió observar gráficamente los cambios entre los estados alto y bajo de la señal.

Objetivos

1. Escribir un blink (directo y por toggle) usando acceso directo a registros
2. Escribir un (directo y por toggle) blink usando SDK
3. Medir el ancho de pulso con el Logic Analyzer

Marco teórico

Los sistemas embebidos son sistemas de computo diseñados para realizar funciones específicas dentro de un producto mayor, que interactúa con el mundo físico mediante sensores y actuadores (1), estos tienen dos componentes, el microcontrolador que se compone del CPU, la memoria y periféricos y el microprocesador que se compone solamente del CPU.

Cada instrucción que ejecuta la CPU tarda un número fijo de ciclos de reloj. El reloj no lleva información si no que lleva el pulso que le da la señal al circuito cuándo dar el siguiente paso al igual que ayuda a la sincronización del chip (1).

El bus es el sistema de comunicación que permite transferir datos, direcciones e instrucciones entre el procesador, la memoria y los periféricos mientras que la memoria almacena tanto las instrucciones del programa como los datos utilizados durante la ejecución. En un microcontrolador se utilizan diferentes tipos de memoria, como la memoria Flash para el código y la memoria RAM para las variables temporales.

Los registros son la memoria interna del microcontrolador donde cada bit controla o refleja el estado de algún elemento del hardware (1).

Los GPIO (General Purpose Input/Output) son pines de entrada y salida de propósito general que permiten interactuar con dispositivos externos, utilizada en microcontroladores, procesadores y circuitos integrados. Un GPIO puede configurarse en estado lógico alto (HIGH) o bajo (LOW) (2).

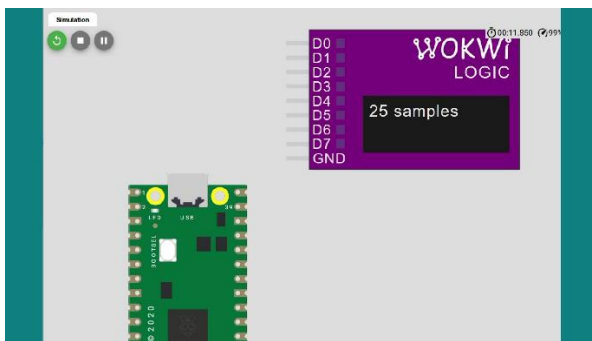
El módulo SIO (Single-cycle Input/Output) es un periférico interno que permite controlar los GPIO mediante acceso directo a registros, realizando operaciones como configurar un pin, encenderlo, apagarlo o cambiar su estado (toggle) en un solo ciclo de reloj (3).

Desarrollo

Materiales

1. Programa wokwi
2. Programa surfer
3. Computadora

Esquemático



Desarrollo

```
> 5to semestre > Elementos programables > Practica 1 > SDK Directo.c
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  int main() {
5      const uint LED_PIN = PICO_DEFAULT_LED_PIN;
6      gpio_init(LED_PIN);
7      gpio_set_dir(LED_PIN, GPIO_OUT);
8      while (true) {
9          gpio_put(LED_PIN, 1); //enciende el led (alto)
10         sleep_ms(500);
11         gpio_put(LED_PIN, 0); //apaga el led (bajo)
12         sleep_ms(500);
13     }
14 }
```

- #include "pico/stdlib.h"

Incluye la biblioteca estándar del SDK de Raspberry Pi Pico

- `Const uint LED_PIN= PICO_DEFAULT_LED_PIN;`

Define una constante llamada LED_PIN que corresponde al GPIO donde está conectado el LED integrado de la Raspberry Pi Pico.

- `gpio_init (LED_PIN);`

Inicializa el GPIO que se utilizará para controlar el LED.

- `gpio_set_dir (LED_PIN, GPIO_OUT)`

Configura el GPIO como una salida, ya que necesitamos enviar una señal para encender y apagar el LED.

- `gpio_put (LED_PIN, 1)`

Coloca el GPIO en estado HIGH (1), haciendo que el LED se encienda o en estado LOW (0) para que se apague el LED.

- `sleep_ms (500)`

Detiene la ejecución durante 500 milisegundos, por lo que el LED permanece encendido durante medio segundo y mantiene el LED apagado durante otros 500 milisegundos.

Resultados

Parámetro	Valor esperado (del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	504.44 ms
Tiempo en bajo	500 ms	494.29 ms
Periodo	1000 ms	998.73 ms
Frecuencia	1 Hz	1.0013 Hz

SDK DIRECTO: No sabemos exactamente qué pasó aquí porque según nosotras le pusimos 500 ms en el código a cada cosa, pero el analizador midió 504.44 ms en alto y 494.29 ms en bajo. Suponemos que como usamos las funciones del SDK (gpio_put), la Raspberry se tarda un poquito en procesar la orden o en mandar la señal antes de hacer el sleep, y por eso el tiempo se descuadró y no dio los 500 ms exactos.

Parámetro	Valor esperado (del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	497.27 ms
Tiempo en bajo	500 ms	500.91 ms
Periodo	1000 ms	998.18 ms
Frecuencia	1 Hz	1.0018 Hz

SDT TOGGLE: En esta parte nos dio 500.91 ms y 497.27 ms, que está bastante cerca de los 500 ms pero sigue sin ser exacto. Sentimos que al usar la función de toggle el código hace menos cosas que en el directo y por eso no se desfasa tanto, pero la verdad no entendimos al cien por qué el simulador no lo pone exacto; nos imaginamos que es un pequeño retraso de Wokwi o de la computadora al correr el programa.

Parámetro	Valor esperado (del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	481.51 ms
Tiempo en bajo	500 ms	503.74 ms
Periodo	1000 ms	985.25 ms
Frecuencia	1 Hz	1.0150 Hz

SIO DIRECTO: Aquí se desfasó bastante más porque el tiempo en alto dio 481.51 ms y el periodo total ni llegó a los 1000 ms (dio 985.25 ms). Se supone que con los registros SIO la placa responde directo al hardware sin pasar por el SDK, pero no sabemos por qué el sleep_ms terminó durando menos tiempo del que le pusimos en el código, suponemos que el temporizador del simulador se aceleró o se trabó un poco en esa parte.

Parámetro	Valor esperado (del código)	Valor medido (Logic Analyzer)
Tiempo en alto	500 ms	493.34 ms
Tiempo en bajo	500 ms	505.94 ms
Periodo	1000 ms	999.27 ms
Frecuencia	1 Hz	1.0007 Hz

SIO TOGGLE: Esta fue la medición más cercana al segundo completo (999.27 ms), aunque individualmente un lado dio 505.94 ms y el otro 493.34 ms. Creemos que al cambiar el bit directamente en el registro con toggle el programa casi no pierde tiempo en instrucciones, pero sigue habiendo una pequeña diferencia de milisegundos que suponemos es culpa del simulador virtual y de cómo lee los pulsos la herramienta.

Evidencias



Video del led.mp4

```

C SIO Directo.c X C SDK Directo.c X C SDK Toggle.c X C SIO Toggle.c X
D: > 5to semestre > Elementos programables > Practica 1 > C SDK Directo.c
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  int main() {
5      const uint LED_PIN = PICO_DEFAULT_LED_PIN;
6      gpio_init(LED_PIN);
7      gpio_set_dir(LED_PIN, GPIO_OUT);
8      while (true) {
9          gpio_put(LED_PIN, 1); //enciende el led (alto)
10         sleep_ms(500);
11         gpio_put(LED_PIN, 0); //apaga el led (bajo)
12         sleep_ms(500);
13     }
14 }

```

```

C SIO Directo.c X C SDK Directo.c X C SDK Toggle.c X C SIO Toggle.c X
D: > 5to semestre > Elementos programables > Practica 1 > C SDK Toggle.c
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  int main() {
5      const uint LED_PIN = PICO_DEFAULT_LED_PIN;
6      gpio_init(LED_PIN);
7      gpio_set_dir(LED_PIN, GPIO_OUT);
8      while (true) {
9          gpio_xor_mask(1u << LED_PIN); // hace toggle del pin mediante SDK
10         sleep_ms(500);
11     }
12 }

```

```

C SIO Directo.c X
D: > 5to semestre > Elementos programables > Practica 1 > C SIO Directo.c
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  int main() {
5      const uint32_t bit = 1u << PICO_DEFAULT_LED_PIN;
6      gpio_init(PICO_DEFAULT_LED_PIN);
7      sio_hw->gpio_oe_set = bit; //habilita la salida
8      while (true) {
9          sio_hw->gpio_set = bit; //enciende el led (alto)
10         sleep_ms(500);
11         sio_hw->gpio_clr = bit; //apaga el led (bajo)
12         sleep_ms(500);
13     }
14 }

```

```

C SIO Directo.c X C SIO Toggle.c X
D: > 5to semestre > Elementos programables > Practica 1 > C SIO Toggle.c
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  int main() {
5      const uint32_t bit = 1u << PICO_DEFAULT_LED_PIN;
6      gpio_init(PICO_DEFAULT_LED_PIN);
7      sio_hw->gpio_oe_set = bit; //habilita la salida
8
9      while (true) {
10         sio_hw->gpio_togl = bit; //invierte automaticamente la función inicial
11         sleep_ms(500);
12     }
13 }

```

Conclusiones

Esta práctica nos permitió comparar las dos formas de programación, acceso directo a registros y SDK, y comprobar mediante el análisis de las señales que ambas pueden producir un comportamiento equivalente, además de comprender de manera más práctica cómo se genera y se mide una señal digital en un microcontrolador.

Bibliografías

[1] «Bare-Metal/O&Timing Inside the MCU: Clocks, Bus, Memory, SIO; Register-level blink», 18 de agosto de 2026.

[2] «GPIO - definición», *Electronic Components*, 1980.
<https://www.tme.com/mx/es/news/library-articles/glossary/page/61888/gpio-general-purpose-inputoutput-definicion/> (accedido 21 de agosto de 2026).

[3] OpenAI. (2026). *ChatGPT* (GPT-5.6 Luna) [Modelo de lenguaje grande].